InfoControl

Desenvolvimento web

Iorran Valença e Gabriel Pereira

Introdução

Com o crescimento do setor de tecnologia e a constante demanda por equipamentos e serviços de informática, torna-se essencial que empresas desse segmento possuam ferramentas eficientes para gerenciar suas operações comerciais. A organização adequada de dados como clientes, produtos, vendas e estoque impacta diretamente na qualidade do atendimento e na tomada de decisões.

Neste contexto, este projeto propõe o desenvolvimento de um sistema web para automatizar e integrar os principais processos de uma loja de informática.

Descrição do Sistema

O sistema **InfoControl** é uma aplicação web voltada para a gestão comercial de uma loja de informática. Ele permitirá o controle de clientes, produtos, estoque e vendas, além de oferecer relatórios gerenciais que auxiliam na análise do desempenho do negócio.

A aplicação contará com uma API REST para comunicação entre backend e frontend, autenticação baseada em token JWT, controle de acesso por perfis de usuário e armazenamento de dados em banco relacional.

Contexto do Negócio

A loja de informática comercializa produtos como computadores, periféricos e acessórios tecnológicos para clientes finais. Muitas vezes, o controle dessas operações é feito de forma manual ou com ferramentas limitadas, o que pode gerar inconsistências, perda de informações e dificuldades no controle de estoque.

O sistema proposto busca centralizar todas essas operações em uma única plataforma, proporcionando maior organização, segurança e eficiência nos processos comerciais.

Objetivo do Sistema

Desenvolver um sistema web que permita gerenciar as operações comerciais de uma loja de informática, incluindo cadastro de clientes, controle de produtos e estoque, registro de vendas, autenticação de usuários e geração de relatórios gerenciais.

Requisitos Funcionais

RF01. O sistema deve permitir o login de usuários.
RF02. O sistema deve permitir o cadastro de clientes.
RF03. O sistema deve permitir a edição de clientes.
RF04. O sistema deve permitir a listagem de clientes.
RF05. O sistema deve permitir a exclusão de clientes, respeitando regras de negócio.
RF06. O sistema deve permitir o cadastro de produtos.
RF07. O sistema deve permitir a edição de produtos.
RF08. O sistema deve permitir a listagem de produtos.
RF09. O sistema deve permitir a exclusão de produtos.
RF10. O sistema deve permitir o registro de vendas.
RF11. O sistema deve permitir adicionar múltiplos itens em uma venda.
RF12. O sistema deve calcular automaticamente o valor total da venda.
RF13. O sistema deve atualizar o estoque após a realização da venda.
RF14. O sistema deve impedir a venda de produtos com estoque insuficiente.
RF15. O sistema deve gerar relatórios de vendas por período.
RF16. O sistema deve permitir consulta de vendas por cliente.
RF17. O sistema deve gerar gráfico anual de vendas.
RF18. O sistema deve controlar acesso por perfil (ADMIN e FUNCIONARIO).

Requisitos Não Funcionais

RNF01. O sistema deve ser desenvolvido utilizando arquitetura em camadas.
RNF02. O sistema deve possuir interface web desenvolvida com Django.
RNF03. O sistema deve disponibilizar API REST com retorno em JSON.
RNF04. O sistema deve utilizar autenticação baseada em token JWT.
RNF05. O sistema deve armazenar senhas de forma criptografada.
RNF06. O sistema deve utilizar banco de dados relacional.
RNF07. O sistema deve ser versionado no GitHub.
RNF08. O sistema deve possuir documentação técnica detalhada.
RNF09. O sistema deve ser organizado de forma a facilitar manutenção e escalabilidade.

Arquitetura Proposta

O sistema será desenvolvido utilizando uma arquitetura em camadas, separando responsabilidades para melhorar a organização e manutenção do código.

As principais camadas são:

Camada de Apresentação: responsável pela interface web desenvolvida em Django.

Camada de API: responsável pelos endpoints REST e comunicação em JSON.

Camada de Negócio: responsável pelas regras de negócio, como cálculo de vendas e controle de estoque.

Camada de Persistência: responsável pela comunicação com o banco de dados.

Essa arquitetura permite maior modularidade, reutilização de código e facilidade de evolução do sistema.

Padrões de Projeto Utilizados

Para garantir organização e boas práticas de desenvolvimento, serão utilizados os seguintes padrões:

Arquitetura em Camadas: separação entre apresentação, negócio e persistência.

MVT (Model-View-Template): padrão utilizado pelo Django na construção da interface web.

Service Layer: centralização das regras de negócio em serviços específicos.

Serializer Pattern: utilizado pelo Django REST Framework para transformação de dados entre objetos e JSON.

Esses padrões contribuem para um sistema mais organizado, testável e de fácil manutenção.

Modelo Conceitual (Descrição do Domínio)

O sistema é composto pelas seguintes entidades principais:

Usuário: responsável pela autenticação e operação do sistema.

Cliente: representa os consumidores da loja.

Produto: representa os itens disponíveis para venda.

Venda: representa uma transação realizada.

Item de Venda: representa os produtos incluídos em uma venda.

Relacionamentos:

Um cliente pode realizar várias vendas.

Uma venda possui vários itens.

Cada item de venda está associado a um produto.

Um usuário registra várias vendas.

Modelo Lógico do Banco de Dados

O banco de dados será composto pelas seguintes tabelas:

usuarios

id (PK)

username

password

perfil

clientes

id (PK)

nome

cpf (único)

email

telefone

endereco

produtos

id (PK)

nome

descricao

preco

quantidade_estoque

- estoque_minimo

vendas

id (PK)

data

cliente_id (FK)

usuario_id (FK)

valor_total

itens_venda

id (PK)

venda_id (FK)

produto_id (FK)

quantidade

preco_unitario

subtotal

Regras de Negócio

Clientes

- CPF não pode ser duplicado.
- Email deve ser válido.
- Cliente não pode ser removido se possuir vendas registradas.

Produtos

- Preço não pode ser negativo.
- O sistema deve controlar estoque mínimo.
- Não permitir venda com estoque insuficiente.

Vendas

- Não permitir vendas sem itens.
- Valor total deve ser calculado automaticamente.
- O estoque deve ser atualizado após a venda.

Usuários

- Senha deve ser criptografada.
- Autenticação deve ser feita via token JWT.
- O sistema deve controlar acesso por perfil (ADMIN e FUNCIONARIO)

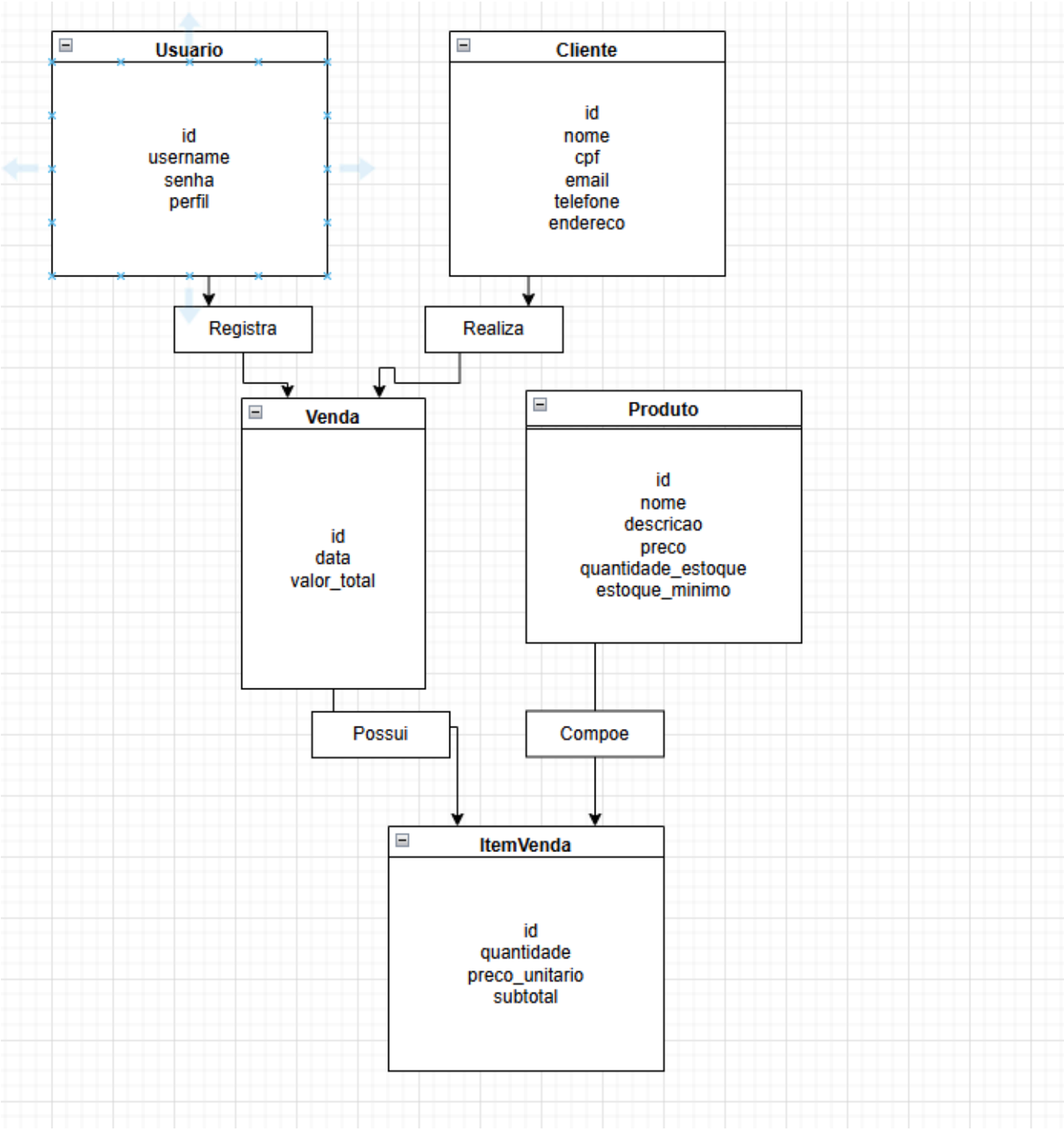

Diagrama de Domínio

O diagrama de domínio representa as principais entidades do sistema e seus relacionamentos, considerando o contexto de uma loja de informática. Ele permite visualizar como clientes, produtos, vendas e usuários se relacionam no negócio

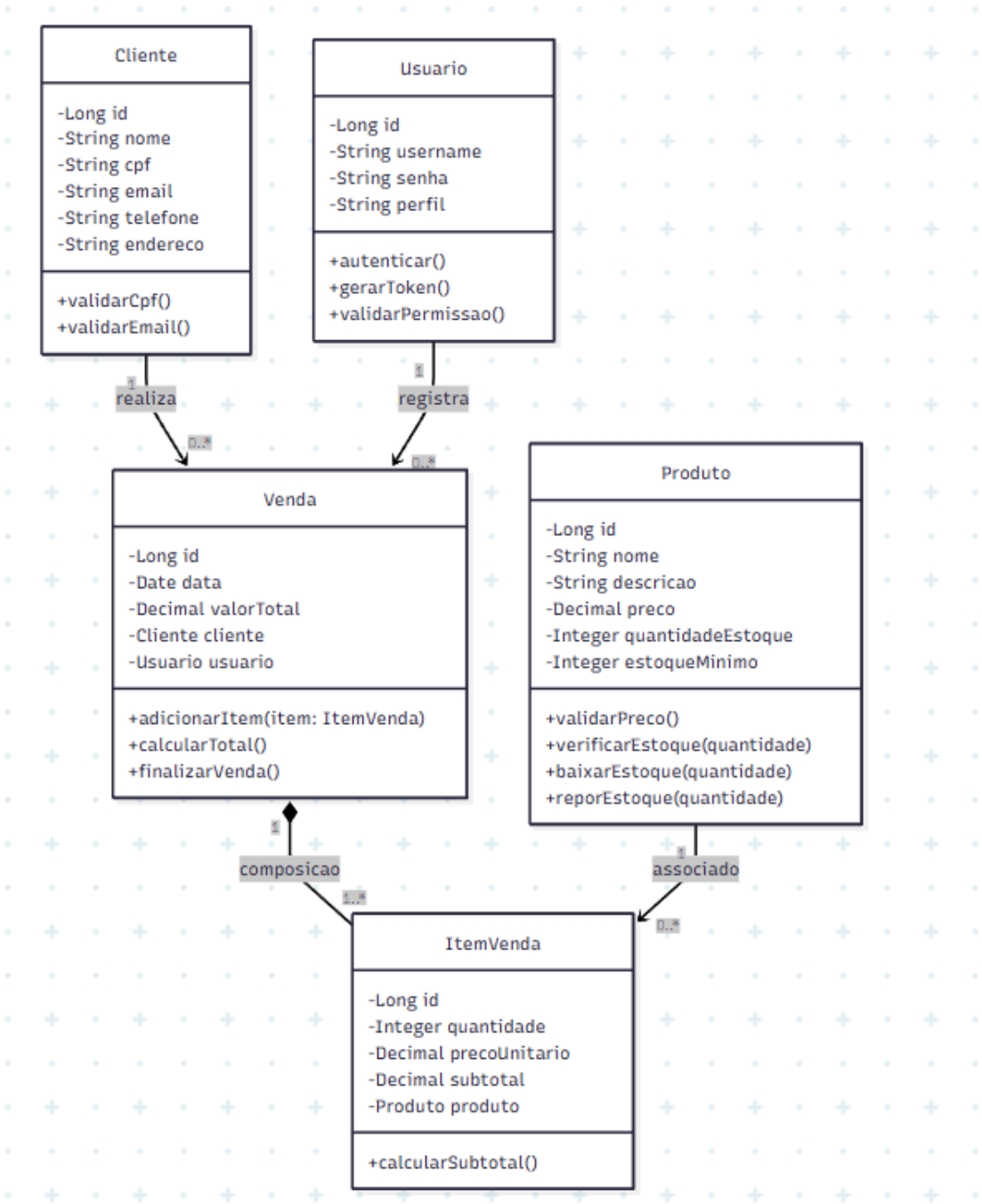

Diagrama de Classes

O diagrama de classes apresenta a estrutura do sistema sob a perspectiva orientada a objetos, incluindo atributos e métodos das classes principais, bem como seus relacionamentos.

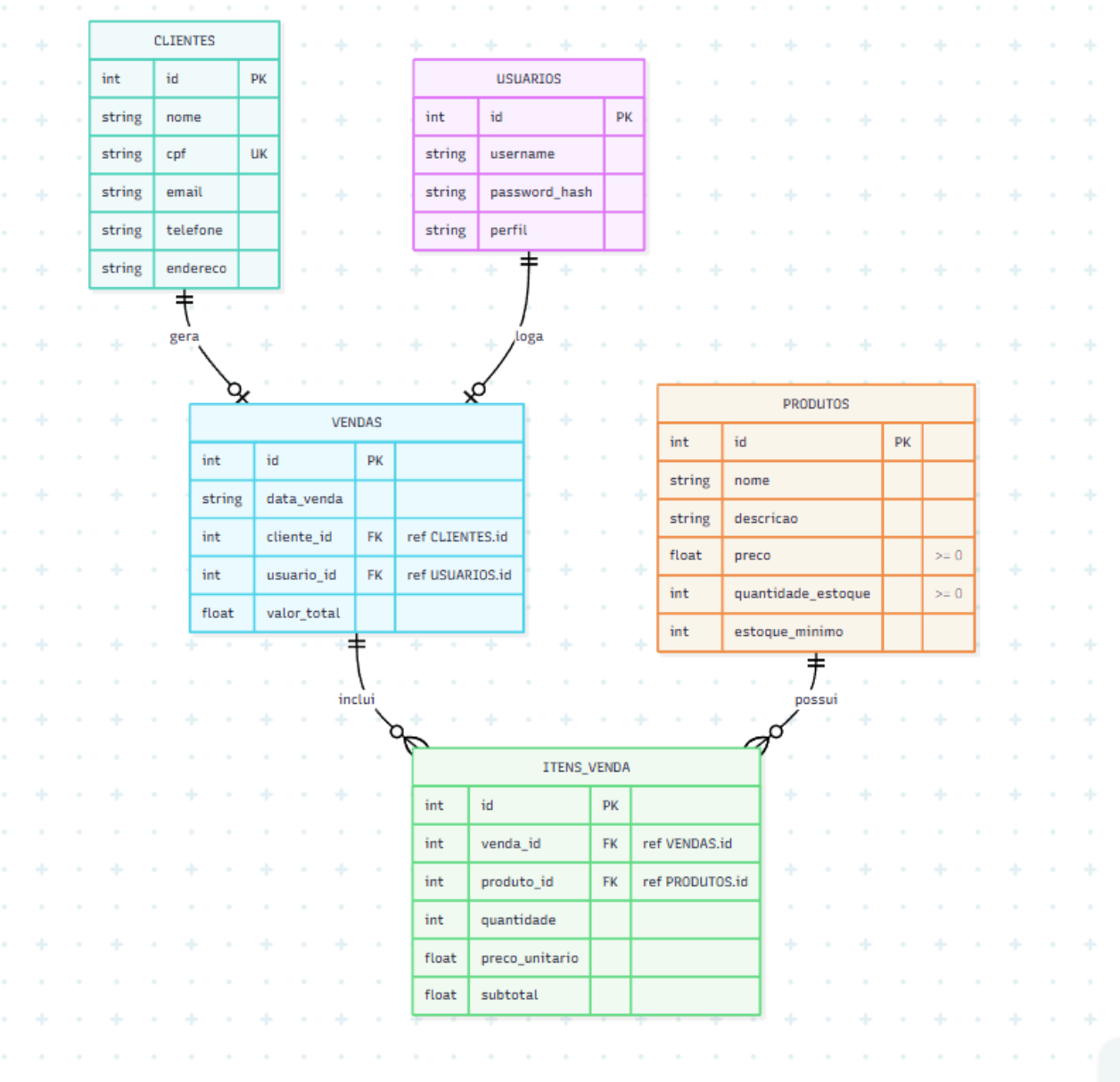

Diagrama lógico do banco de dado

O diagrama lógico do banco de dados apresenta a estrutura das tabelas responsáveis pela persistência das informações do sistema, incluindo clientes, usuários, produtos, vendas e itens de venda. Foram definidas chaves primárias para identificação única dos registros e chaves estrangeiras para garantir a integridade referencial entre as entidades.

A modelagem contempla as principais regras de negócio, como o relacionamento entre clientes e vendas, o vínculo entre usuários e registros de venda, e a associação entre produtos e itens de venda. Além disso, foram consideradas restrições como unicidade

do CPF, controle de estoque e consistência dos valores monetários, assegurando a confiabilidade dos dados armazenados.

Script do banco

O script de criação do banco de dados foi elaborado com base no diagrama lógico do sistema, contemplando as tabelas principais, suas chaves primárias, chaves estrangeiras e restrições de integridade. Foram incluídas validações básicas diretamente no banco de dados, como unicidade do CPF, controle de valores não negativos e domínio do campo perfil, garantindo maior consistência no armazenamento das informações.

```
CREATE TABLE usuarios (
    id BIGINT PRIMARY KEY GENERATED ALWAYS AS IDENTITY,
    username VARCHAR(50) NOT NULL UNIQUE,
    password_hash VARCHAR(255) NOT NULL,
    perfil VARCHAR(20) NOT NULL,
    CONSTRAINT chk_usuario_perfil
        CHECK (perfil IN ('ADMIN', 'FUNCIONARIO'))
);

CREATE TABLE clientes (
    id BIGINT PRIMARY KEY GENERATED ALWAYS AS IDENTITY,
    nome VARCHAR(100) NOT NULL,
    cpf VARCHAR(14) NOT NULL UNIQUE,
    email VARCHAR(100) NOT NULL,
    telefone VARCHAR(20),
    endereco VARCHAR(150),
    CONSTRAINT chk_cliente_email
```

```
        CHECK (POSITION('@' IN email) > 1)
);

CREATE TABLE produtos (
    id BIGINT PRIMARY KEY GENERATED ALWAYS AS IDENTITY,
    nome VARCHAR(100) NOT NULL,
    descricao VARCHAR(255),
    preco NUMERIC(10,2) NOT NULL,
    quantidade_estoque INT NOT NULL DEFAULT 0,
    estoque_minimo INT NOT NULL DEFAULT 0,
    CONSTRAINT chk_produto_preco
        CHECK (preco >= 0),
    CONSTRAINT chk_produto_quantidade
        CHECK (quantidade_estoque >= 0),
    CONSTRAINT chk_produto_estoque_minimo
        CHECK (estoque_minimo >= 0)
);

CREATE TABLE vendas (
    id BIGINT PRIMARY KEY GENERATED ALWAYS AS IDENTITY,
    data_venda TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    cliente_id BIGINT NOT NULL,
    usuario_id BIGINT NOT NULL,
    valor_total NUMERIC(10,2) NOT NULL DEFAULT 0,
    CONSTRAINT fk_vendas_cliente
        FOREIGN KEY (cliente_id) REFERENCES clientes(id),
    CONSTRAINT fk_vendas_usuario
```

```
        FOREIGN KEY (usuario_id) REFERENCES usuarios(id),
    CONSTRAINT chk_venda_valor_total
        CHECK (valor_total >= 0)
);

CREATE TABLE itens_venda (
    id BIGINT PRIMARY KEY GENERATED ALWAYS AS IDENTITY,
    venda_id BIGINT NOT NULL,
    produto_id BIGINT NOT NULL,
    quantidade INT NOT NULL,
    preco_unitario NUMERIC(10,2) NOT NULL,
    subtotal NUMERIC(10,2) NOT NULL,
    CONSTRAINT fk_itens_venda_venda
        FOREIGN KEY (venda_id) REFERENCES vendas(id) ON DELETE CASCADE,
    CONSTRAINT fk_itens_venda_produto
        FOREIGN KEY (produto_id) REFERENCES produtos(id),
    CONSTRAINT chk_item_quantidade
        CHECK (quantidade > 0),
    CONSTRAINT chk_item_preco_unitario
        CHECK (preco_unitario >= 0),
    CONSTRAINT chk_item_subtotal
        CHECK (subtotal >= 0)
);
```